NATURAL LANGUAGE

TO DATA FILTERING & VISUALIZATION

Gaurav Kumar

Data Analyst | ML Engineer | Python Developer

Python	Streamlit	Groq LLM
Pandas	Scikit-learn	Matplotlib

Problem Statement

- Non-technical users struggle to query and analyze data without SQL or Python
- Manual filtering in spreadsheets is slow, error-prone, and doesn't scale
- Creating charts requires technical knowledge of plotting libraries
- Business teams are blocked waiting for analysts to run basic queries
- No existing simple tool that bridges plain English ↔ live dataset operations

Solution Overview

- Type a plain-English query → system converts it to a Pandas filter expression
- Groq LLM handles query understanding with high accuracy (91% on 200+ queries)
- Safety layer validates every generated expression before execution
- Filtered results displayed as tables; visualization queries rendered as charts

Key Features

Upload CSV / Excel

Upload dataset, auto column detection

NL Query Input

Type plain English — no code required

Smart Filtering

Pandas expression generated + safely executed

Chart Generation

Bar, line, pie, scatter, histogram, box

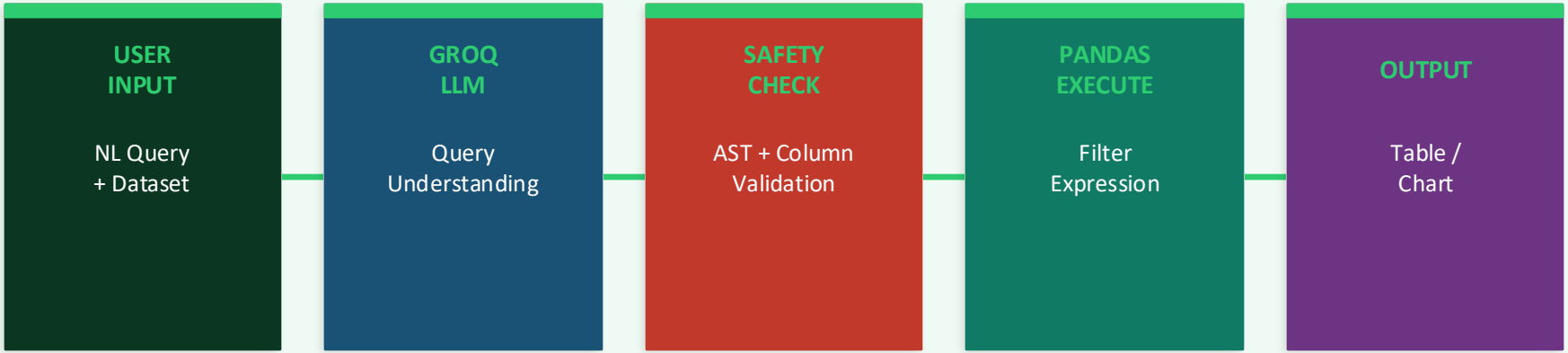
ML Pipeline

Select target column → train sklearn model

Safety First

eval environment

System Architecture



Query Routing after LLM

- **Filtering Query:** *e.g. "salary > 50k" → df[expression] → Results Table*
- **Visualization Query:** *e.g. "plot age dist" → JSON chart spec → Chart render*
- **ML Query:** *Target column given → sklearn Pipeline → Model metrics*

LLM Integration — Groq

- Groq LLM is the AI engine: fast, low-latency, API-accessible via .env key
- Filtering: converts plain English → safe pandas expression string
- Visualization: returns structured JSON spec (chart type + column list)
- ML: detects intent when user specifies a "target column" for training
- Achieves 91% query interpretation accuracy tested on 200+ real queries

Safety Mechanism

⚠ **LLM output is NEVER executed directly — every expression goes through a strict validation pipeline**

01

Prefix Check

Expression must start with df[
(rejects any code that doesn't filter the dataframe)

02

Column Allowlist

Only columns present in the
uploaded file are permitted
(no injection of phantom fields)

03

AST Allowlist

Python AST parse — only comparisons, booleans, numbers, strings
(no imports, calls, or attributes)

04

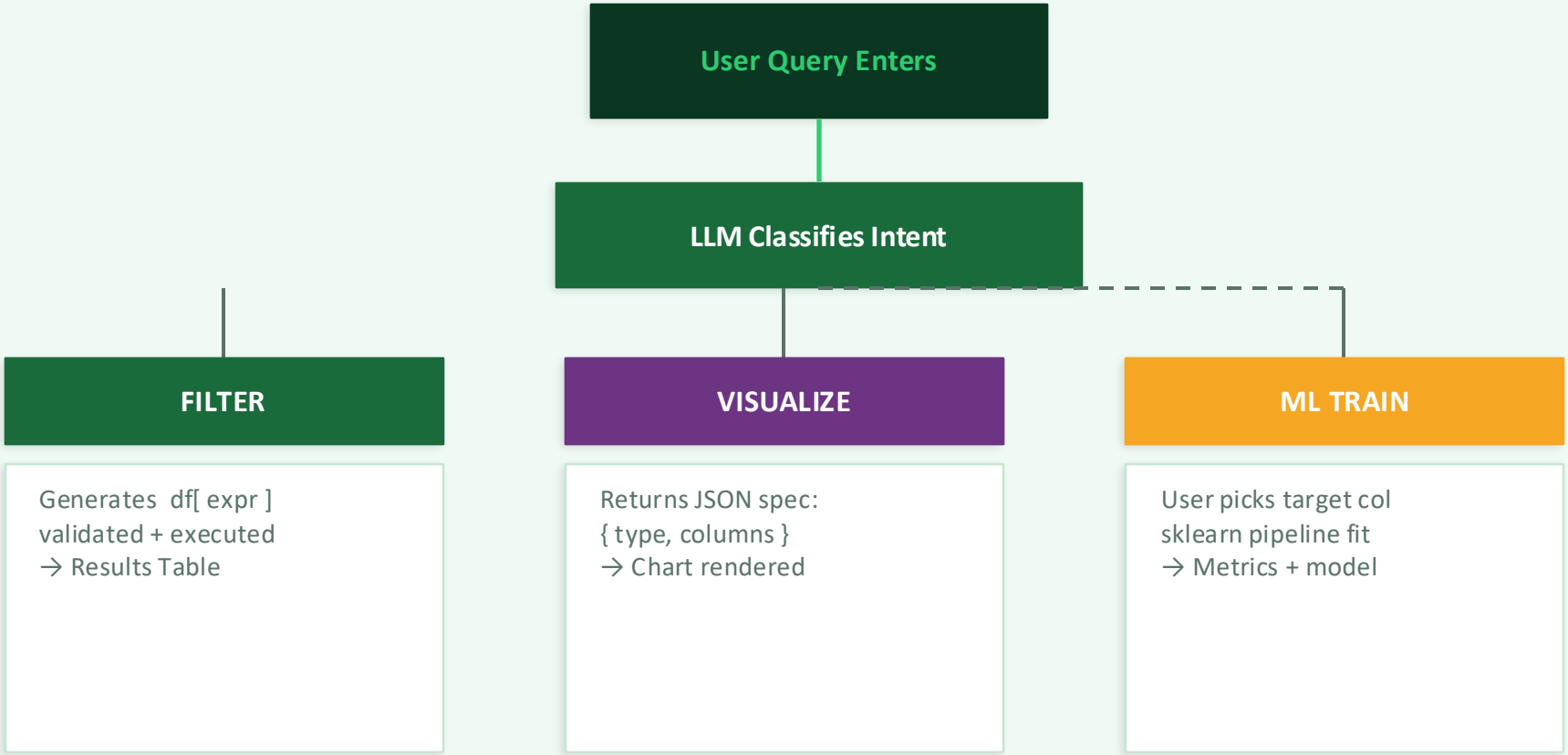
Restricted eval()

Executes in isolated env: no builtins, no globals
(sandboxed execution context)

Data Processing

- User uploads CSV or Excel → loaded into pandas DataFrame automatically
- Column names extracted and sent to LLM as context for accurate query mapping
- Query transformation: NL string → pandas filter string via LLM
- Expression validated (safety layer) then executed with df[expression]
- Supports 50,000+ row datasets with performant in-memory filtering

Query Routing Logic



Visualization System

LLM JSON Response Example

```
{
  "type": "bar",
  "columns": ["region", "revenue"],
  "title": "Revenue by Region"
}
```

Supported Chart Types

- Bar
- Line
- Pie
- Histogram
- Scatter
- Box

All charts rendered via safe predefined templates — no raw code execution

Machine Learning — Optional Feature

- User selects a target column from the uploaded dataset
- Sklearn Pipeline auto-configured: preprocessing + model training
- Supports classification and regression depending on target type
- Outputs: accuracy, F1, confusion matrix, feature importances
- End-to-end — from raw CSV to trained model in a single click

Demo Flow

01

Upload File

CSV or Excel
loaded into pandas

02

Type Query

"show top 5
sales by region"

03

LLM Converts

pandas expr
generated

04

Validate

Safety check
passes / rejects

05

View Output

Table or
Chart displayed

```
streamlit run main.py   →   localhost:8501   →   Upload → Query → Analyze
```

Example: "Show top 10 products by revenue" → df.nlargest(10, "revenue") → Table + Bar Chart

Example Queries

Type	Natural Language Query	Generated Expression
FILTER	"Show employees with salary > 50k"	`df[df["salary"] > 50000]`
FILTER	"Top 10 products by revenue"	`df.nlargest(10, "revenue")`
FILTER	"Customers from Mumbai aged below 30"	`df[(df["city"]=="Mumbai") & (df["age"]<30)]`
VISUALIZE	"Plot age distribution"	`{ type: "hist", columns: ["age"] }`
VISUALIZE	"City-wise user count"	`{ type: "bar", columns: ["city", "count"] }`

Challenges Faced

Technical Challenges

- Mapping ambiguous NL to correct pandas syntax reliably
- AST validation covering all edge cases without blocking valid queries
- Handling diverse CSV structures (mixed types, missing values, special chars)
- Groq API latency for long queries on large datasets

Product Challenges

- Communicating errors clearly when LLM output fails validation
- Visualization spec ambiguity (LLM picking wrong chart type)
- Balancing safety strictness vs. flexibility for complex queries
- Ensuring ML pipeline works across different data shapes and targets

Future Improvements

- Upgrade to embedding-based models (RAG) for higher query accuracy
- Live database support: connect to MySQL, PostgreSQL, or BigQuery directly
- Multi-turn conversation: refine filters interactively ("now add age > 25")
- Deploy as cloud web app (AWS / GCP) with user auth and saved query history
- Expand ML pipeline: AutoML, model explainability (SHAP), hyperparameter search
- Support for aggregations: GROUP BY, pivot tables, advanced window functions

Conclusion

- Democratizes data analysis — anyone can query and chart without code
- Uniquely combines Groq LLM + strict safety layer + predefined chart templates
- Tested on 50,000-row datasets with 91% NL query interpretation accuracy
- Reduces analysis time for non-technical users by an estimated 60%
- Demonstrates real-world LLM + Data Science + Web integration end-to-end

Thank You

Gaurav Kumar | gauravbaba311@gmail.com | github.com/GauravBaba3